UNIVERSITY OF INFORMATION TECHNOLOGY AND SCIENCES

Department of Computer Science and Engineering

FINAL DIAGRAMS

CampusConnect

University Club, Event & Activity Management Platform

Course	Software Project Design and Development
Course Code	416
Submitted By	Md. Shakhawat Hossain
Student ID	0432320005101009
Batch	54
Submitted To	Al Imtiaz, Course Teacher

Department of Computer Science and Engineering
University of Information Technology and Sciences

Table of Contents

1. Introduction and Scope

2. High-Level System Architecture

3. Use Case Diagram

4. Entity Relationship Diagram (ERD)

5. Event Registration Activity Diagram

6. Event Approval State Diagram

7. Registration Sequence Diagram

8. Design Notes and Assumptions

9. Conclusion

1. Introduction and Scope

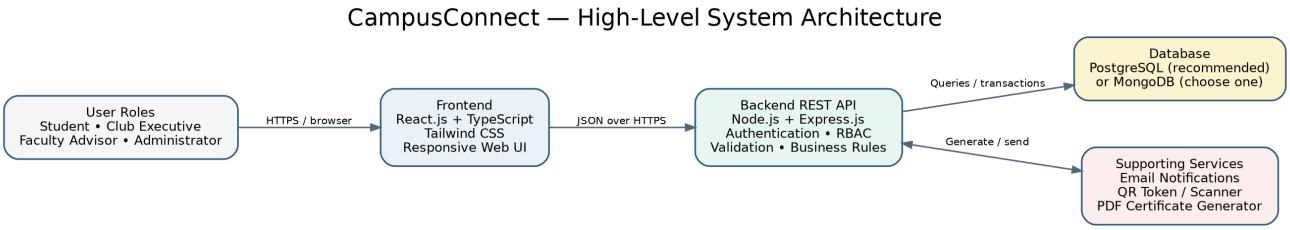
This assignment presents the final system diagrams for CampusConnect, a web-based university club, event and activity management platform. The diagrams are derived from the supplied Software Requirements Specification (SRS). They describe the system structure, user interactions, core data relationships, event registration workflow and event approval lifecycle.

The SRS identifies four principal roles: Student, Club Executive, Faculty Advisor and University Administrator. Planned technologies include React.js, TypeScript and Tailwind CSS for the frontend; Node.js and Express.js for REST APIs; and PostgreSQL or MongoDB for persistence. The SRS leaves the final database choice open, so PostgreSQL is shown as a recommended option rather than a confirmed implementation decision.

Purpose of the diagrams

• Communicate the system design clearly to the teacher and development team.
• Connect user roles and workflows to the requirements in the SRS.
• Provide a reference for implementation, testing and later refinement.

2. High-Level System Architecture

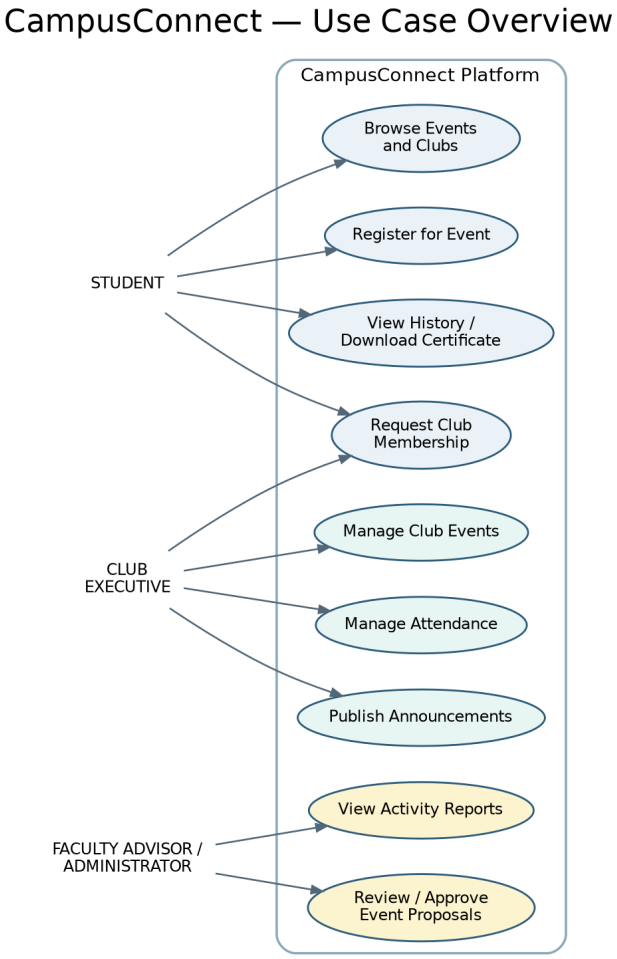


The architecture separates the browser-based user interface, backend API, persistent database and supporting services. The frontend communicates with the Express REST API using JSON; production communication should use HTTPS. The backend applies authentication, role-based authorization and server-side validation before changing data.

Key points

• User roles access the system through a responsive web interface.
• The API is responsible for business rules and access checks; hiding a button in the UI is not sufficient security.
• Email delivery, QR-token generation/scanning and PDF certificate generation are supporting capabilities.
• Select one database before implementation and document the chosen libraries and configuration.

3. Use Case Diagram

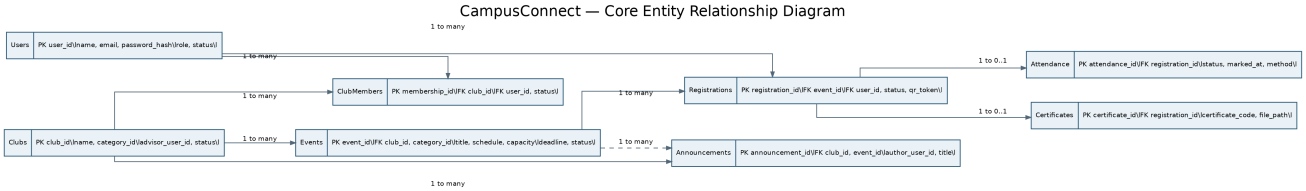


The use case diagram maps major platform capabilities to the actors defined in the SRS. Students discover events and clubs, register for events, request membership and access their own history and certificates. Club executives manage events, attendance and announcements for assigned clubs. Faculty advisors and administrators review event proposals and view reports within their authorization scope.

Key points

• Only published events should appear in the public event directory.
• Executives may manage only their assigned club(s).
• Private student records and reports must be protected by role and resource scope.

4. Entity Relationship Diagram (ERD)

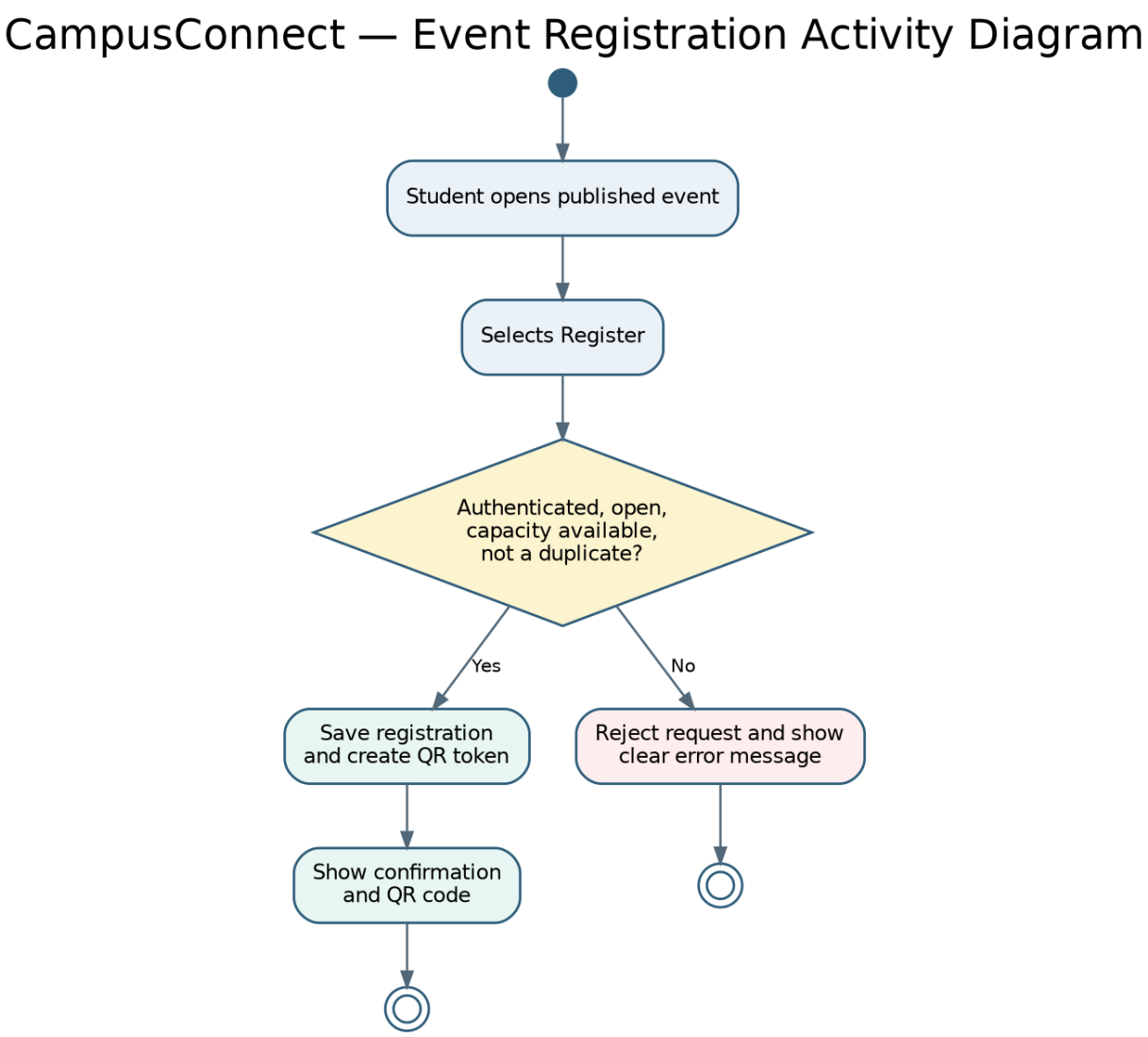


The ERD summarizes the core entities and relationships described in the SRS. Users and clubs are connected through ClubMembers, which resolves the many-to-many membership relationship. Events belong to clubs; Registrations connect students to events. Attendance and Certificates are associated with a registration, while Announcements may relate to a club and/or event.

Key points

• PK means primary key; FK means foreign key.
• Prevent duplicate active registrations and duplicate attendance records using database constraints and server-side checks.
• Certificate codes should be unique and certificates should only be linked to eligible registrations.
• The SRS also lists EventCategories, Notifications and optional Feedback; these are omitted from this simplified core ERD for readability.

5. Event Registration Activity Diagram

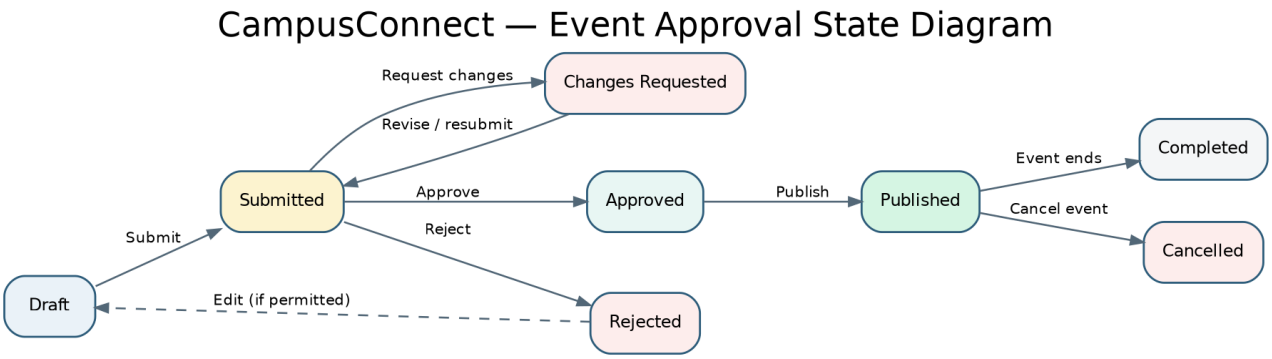


This activity diagram follows the registration workflow in UC-01. After a student selects Register, the backend validates authentication, event status, registration deadline, capacity and duplicate registration. Only a valid request is saved; the system then returns confirmation and a QR token. Invalid requests are rejected with an understandable message.

Key points

Important validation rules: no registration after the deadline, no registration when capacity is full, and no duplicate active registration for the same student and event.

6. Event Approval State Diagram

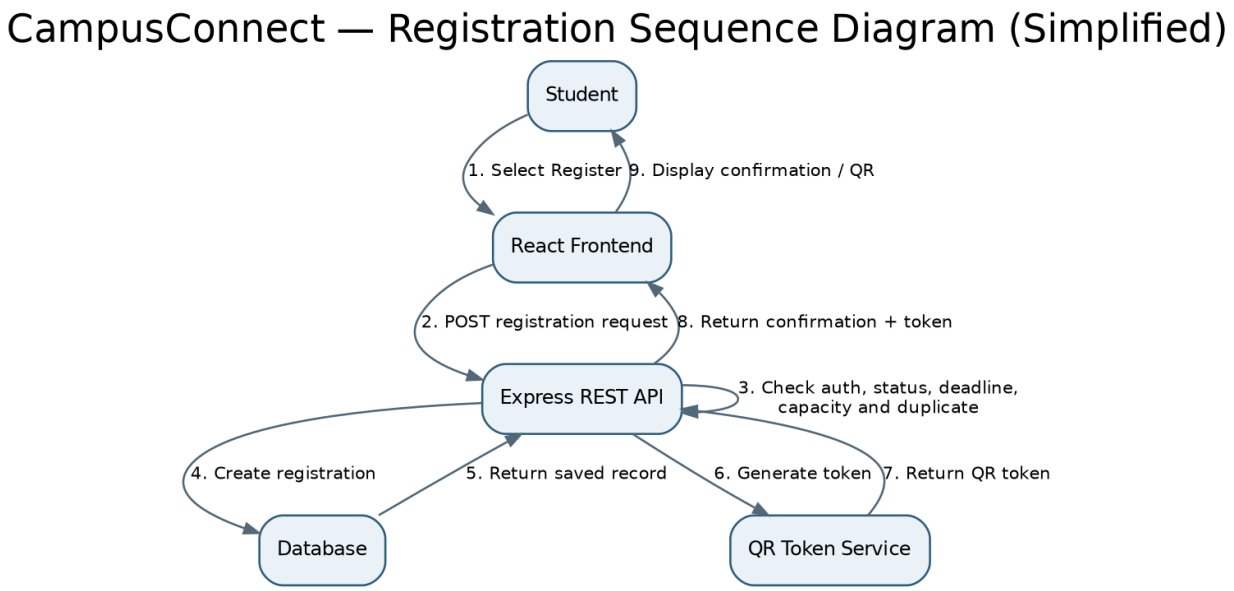


The state diagram describes the event proposal lifecycle. An executive prepares a draft and submits it for review. An advisor or administrator can approve, reject or request changes. Approved events can be published; published events can later be completed or cancelled. A rejected proposal may return to Draft only if editing is permitted by the implementation rules.

Key points

The SRS requires event proposals to be approved before public publication. Decision notes and approval history should be retained, including the actor and timestamp.

7. Registration Sequence Diagram



The sequence diagram shows the order of messages in a successful event-registration request. The frontend sends the request to the API; the API validates the request, persists the registration, requests a QR token and returns a confirmation to the frontend. This is a simplified logical sequence and does not prescribe exact implementation classes or internal library calls.

Key points

If validation fails, the API should return a safe error response and no registration should be created. Database transactions or equivalent safeguards should protect against race conditions involving event capacity.

8. Design Notes and Assumptions

1. Database selection: The SRS permits PostgreSQL or MongoDB and does not record a final choice. PostgreSQL is shown as a recommended default because the platform has related records and integrity constraints; this remains a decision to confirm.

2. Authentication and authorization: The SRS proposes JWT-based authentication and requires server-side role and resource-ownership checks. Passwords must be stored as modern hashes, not plain text.

3. QR attendance: QR tokens are planned to identify registrations. Invalid, reused or unrelated tokens must be rejected; manual attendance entry remains a fallback for authorized organizers.

4. Certificates: PDF certificates are issued only to eligible attendees and can be downloaded by the student associated with the account, subject to authorization.

5. Diagram status: These are clean, final-style diagrams based on the provided SRS, not screenshots of an already implemented system. Update them if the actual implementation or approved requirements differ.

6. Scope: Online payments, native mobile apps, biometric attendance, location tracking and official university SSO/SIS integration are outside the initial release scope in the SRS.

9. Conclusion

The diagrams provide a coherent visual summary of CampusConnect's proposed architecture, user capabilities, data structure and key workflows. Together, they support the SRS by making the system boundaries, permission responsibilities, data relationships and validation steps easier to understand. Before implementation, the team should confirm the database choice, approval rules and any university-specific policies, then keep the diagrams synchronized with the final design.

Source basis: CampusConnect Software Requirements Specification (Version 1.0 draft, dated 13 October 2026), supplied by the student. No external requirements were added.